

RNNs, Attention & Transformers: From Sequential Memory to Parallel Power

KAUST–Mawhiba IOAI Training

Week 3 · Day 17



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

1. Why Sequences Matter
2. RNN Foundations
3. RNN Types & Tasks
4. RNN in PyTorch
5. The Vanishing Gradient Problem
6. LSTM: Long Short-Term Memory
7. GRU: Gated Recurrent Unit
8. LSTM in PyTorch
9. The Attention Mechanism
10. Attention in Seq2Seq (LSTM)
11. Self-Attention & Transformers
12. The Transformer Architecture

Part 1: **Why Sequences Matter**

- ▶ **Day 13:** We learned that words can be represented as *vectors* (Word2Vec, GloVe)
- ▶ **Day 14:** We *visualized* those vectors with PCA, t-SNE, UMAP
- ▶ **Day 16:** We *grouped* similar embeddings with clustering
- ▶ **Today:** We ask — does the *order* of words matter?

Think About It

“The cat chased the dog” vs “The dog chased the cat”

Same words, same embeddings, **completely different meaning!**

- ▶ Word2Vec gives “bank” the **same vector** everywhere
- ▶ But meaning depends on *context*:
 - “I went to the **bank** to deposit money” (financial)
 - “I sat on the river **bank**” (nature)
- ▶ MLPs treat each input independently — they have *no memory*
- ▶ We need a model that can **read word by word** and **remember** what came before

Text:

- ▶ “The cat sat on the mat”
- ▶ Each word depends on previous words

Time Series:

- ▶ Temperature: 30° , 32° , 35° , ?
- ▶ Stock prices over time

Speech:

- ▶ Sound waves over time
- ▶ Order of sounds = meaning

Music:

- ▶ Notes played in sequence
- ▶ Rearrange = different song

Key insight: Order matters! We need models that process sequences.

Running Example: Next-Word Prediction

Throughout today, we'll use a simple task:

Given: "The cat sat on the"

Predict: "mat"

We'll see how RNNs, LSTMs, Attention, and Transformers each handle this!

- ▶ This is called *language modeling* — predicting the next word
- ▶ It's the foundation behind ChatGPT, Google Translate, and autocomplete
- ▶ We'll also explore *sentiment analysis*:
"This movie was amazing" → Positive ✓

1. **RNN Foundations** — Teaching a network to read word by word
2. **RNN Types & Tasks** — Different shapes for different problems
3. **RNN in PyTorch** — Build and train one!
4. **LSTM & GRU** — Solving the “forgetful” RNN problem
5. **LSTM in PyTorch** — Build and train one!
6. **Attention Mechanism** — Let the model focus on what matters
7. **Attention in Seq2Seq** — Supercharging translation
8. **Transformers** — The architecture that changed everything
9. **Transformer in PyTorch** — Build an encoder from scratch!

Part 2: Recurrent Neural Networks

- ▶ A neural network with a *loop* — it passes information from one step to the next
- ▶ Unlike an MLP, an RNN has *memory*: it remembers what it has seen before
- ▶ Think of it like **reading a book word by word**:
 - After reading “The cat sat”, you remember the subject is a cat
 - This memory helps you understand the next word

Key Idea

An RNN processes inputs **one at a time, in order**, and maintains a **hidden state** (memory) that gets updated at each step.

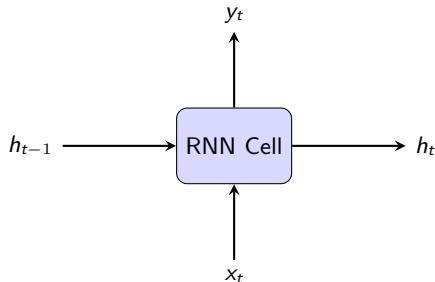
Three components:

- ▶ **Input** x_t : The current word (as a vector)
- ▶ **Hidden state** h_t : The “memory” — a summary of everything seen so far
- ▶ **Output** y_t : The prediction at this step

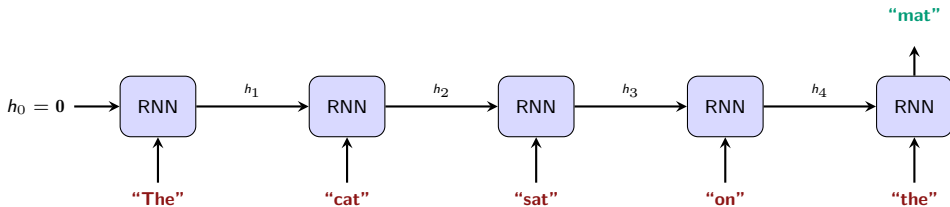
The update rule:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

- ▶ W_{xh} : how to use the new input
- ▶ W_{hh} : how to use the old memory
- ▶ $\tanh$: squashes values to $[-1, 1]$



The same cell is reused
at every time step!



- ▶ It's the **same network** copied for each time step (same weights!)
- ▶ The hidden state h_t carries information forward like a *conveyor belt*
- ▶ After reading all 5 words, h_5 contains a summary of the entire sentence
- ▶ We use h_5 to predict the next word: **"mat"**

Processing “The cat sat on the”

1. $h_0 = [0, 0, 0, \dots]$ (empty memory)
2. Read “The” $\rightarrow h_1 = \tanh(W_{xh} \cdot \text{embed}(\text{“The”}) + W_{hh} \cdot h_0)$
3. Read “cat” $\rightarrow h_2 = \tanh(W_{xh} \cdot \text{embed}(\text{“cat”}) + W_{hh} \cdot h_1)$
4. Read “sat” $\rightarrow h_3$ now “knows” about the cat sitting
5. Read “on” $\rightarrow h_4$ knows something is on something
6. Read “the” $\rightarrow h_5$ ready to predict what comes next

Prediction: $\hat{y} = \text{softmax}(W_{hy} \cdot h_5) \rightarrow$ highest probability: *“mat”*

- ▶ **Same weights** W_{xh} , W_{hh} , W_{hy} are used at *every time step*
- ▶ This means:
 - The model can handle **any length** input
 - 5 words or 500 words — same number of parameters!
 - What the model learns at position 3, it can apply at position 300
- ▶ Compare to an MLP:
 - MLP needs a **fixed input size** (e.g., exactly 10 words)
 - Different weights for each position — no sharing

Parameter sharing = efficiency + generalization to any sequence length

- ▶ **Forward pass:** Process each word, compute loss at the end
- ▶ **Loss:** Cross-entropy between predicted word and actual word

$$\mathcal{L} = - \sum_t \log P(y_t^{\text{true}} | x_1, \dots, x_t)$$

- ▶ **Backward pass:** Gradients flow *backward through all time steps*
- ▶ This is called *Backpropagation Through Time* (BPTT)

Analogy

Imagine you made a mistake in a recipe. BPTT traces back through every step to find where the mistake started — like rewinding a video!

Advantages:

- ✓ Can process inputs of **any length**
- ✓ **Shares parameters** — efficient and generalizable
- ✓ Hidden state captures **context** from previous words
- ✓ Natural fit for **sequential data** (text, speech, time series)

Disadvantages:

- ✗ **Slow:** Must process words *one at a time* (no parallelism)
- ✗ **Forgetful:** Struggles to remember information from many steps ago
- ✗ **Vanishing gradients:** Hard to train on long sequences

Part 3: RNN Types & Tasks

- ▶ Not all sequence tasks have the same shape!
- ▶ **Input:** could be one item or many
- ▶ **Output:** could be one item or many
- ▶ RNNs can be configured in different ways:

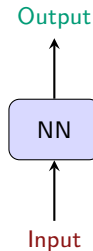
Type	Input	Output
One-to-One	Single	Single
One-to-Many	Single	Sequence
Many-to-One	Sequence	Single
Many-to-Many	Sequence	Sequence

- ▶ Just a regular neural network — no sequence!
- ▶ **Input:** one item → **Output:** one item
- ▶ Example: image → class label

Example

Input: a photo of a cat

Output: “cat”

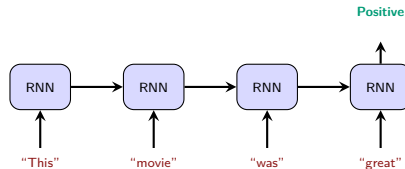


- ▶ Read an entire sequence, produce **one output**
- ▶ The final hidden state summarizes everything

Sentiment Analysis

Input: "This movie was **amazing!**"

Output: **Positive** ✓



Other examples:

- ▶ Email → spam or not spam
- ▶ Patient symptoms → diagnosis

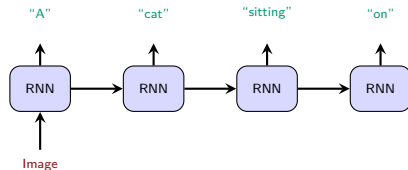
- ▶ One input produces a **whole sequence**
- ▶ Each output becomes the input for the next step

Image Captioning

Input: Photo of a cat on a mat
Output: "A cat sitting on a colorful mat"

Other examples:

- ▶ Music note → full melody
- ▶ Topic word → paragraph



- ▶ Input sequence $\rightarrow$ Output sequence
- ▶ Two variants:
 - **Aligned:** Same length (e.g., POS tagging)
 - **Unaligned:** Different length (e.g., translation)

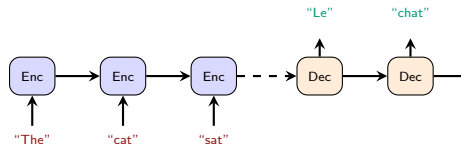
Machine Translation

Input: "The cat sat" (English, 3 words)

Output: "Le chat assis" (French, 3 words)

But: "I am happy" $\rightarrow$ "Je suis content" (same)

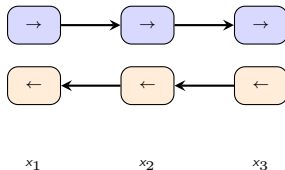
"I am a student" $\rightarrow$ "Ana talib" (shorter!)



Type	Task	Example
One-to-One	Classification	Image → label
Many-to-One	Sequence classification	Review → sentiment
One-to-Many	Sequence generation	Image → caption
Many-to-Many	Seq-to-seq (aligned)	Words → POS tags
Many-to-Many	Seq-to-seq (unaligned)	English → French

- ▶ **Key takeaway:** The same RNN building block can be arranged in different ways to solve very different problems!
- ▶ Choose the architecture based on your *input/output shapes*

- ▶ Regular RNNs only read *left to right*
- ▶ But sometimes future context matters too!
- ▶ **Example:** “The _____ is barking loudly”
 - Knowing “barking” comes later helps predict “dog”
- ▶ *Bidirectional RNN:* Run two RNNs — one forward, one backward
- ▶ Concatenate their hidden states: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$



Part 4: RNN in PyTorch

```
import torch
import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden_dim,
                           batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x):
        # x shape: (batch, seq_len) -- word indices
        emb = self.embedding(x)           # (batch, seq, embed)
        out, h_n = self.rnn(emb)          # out: all hidden states
        last = out[:, -1, :]              # last hidden state
        logits = self.fc(last)            # predict next word
        return logits
```

- ▶ `nn.Embedding`: converts word index $\rightarrow$ dense vector
- ▶ `nn.RNN`: the recurrent layer (processes sequence)
- ▶ `nn.Linear`: maps hidden state $\rightarrow$ vocabulary scores

```
# Toy data: "the cat sat on the mat"
words = ["the", "cat", "sat", "on", "the", "mat"]
word2idx = {w: i for i, w in enumerate(set(words))}
vocab_size = len(word2idx)

# Input: first 5 words, Target: last word
x = torch.tensor([[word2idx[w] for w in words[:-1]]])
y = torch.tensor([word2idx[words[-1]]])

# Create model and train
model = SimpleRNN(vocab_size, embed_dim=16, hidden_dim=32)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(100):
    logits = model(x)
    loss = loss_fn(logits, y)
    optimizer.zero_grad()
    loss.backward()      # BPTT happens here!
    optimizer.step()
```

```
# After training, make a prediction
model.eval()
with torch.no_grad():
    logits = model(x)
    predicted_idx = logits.argmax(dim=1).item()

idx2word = {i: w for w, i in word2idx.items()}
print(f"Predicted next word: {idx2word[predicted_idx]}")
# Output: "Predicted next word: mat"
```

Full Pipeline Recap

1. **Tokenize:** words $\rightarrow$ indices
2. **Embed:** indices $\rightarrow$ dense vectors
3. **RNN:** process sequence, get final hidden state
4. **Linear:** hidden state $\rightarrow$ vocabulary scores
5. **Loss:** cross-entropy between prediction and true word
6. **Backprop:** BPTT through all time steps

Part 5: **Why RNNs Forget**

Try This

“The **cat**, who was sitting on a beautiful red cushion in the corner of the large sunny room next to the window, suddenly _____ up.”

What word goes in the blank? You need to remember “cat” from 20 words ago!

- ▶ Humans can remember important words from far back
- ▶ Vanilla RNNs *cannot* — they forget after just a few steps
- ▶ This is called the *vanishing gradient problem*

- ▶ During BPTT, gradients are **multiplied** at each time step:

$$\frac{\partial \mathcal{L}}{\partial W} \propto \prod_t \frac{\partial h_t}{\partial h_{t-1}}$$

- ▶ Each factor involves $\tanh'$ (derivative of $\tanh$) $\times W_{hh}$
- ▶ If these factors are < 1 , the product *shrinks exponentially*:

$$0.5 \times 0.5 \times 0.5 \times 0.5 \times 0.5 = 0.03125$$

- ▶ After 20 steps: $0.5^{20} \approx 0.000001$ — the gradient is basically **zero**!
- ▶ The network **can't learn** from early words in the sequence

- ▶ If the factors are > 1 , gradients *explode*:

$$2 \times 2 \times 2 \times 2 \times 2 = 32 \quad 2^{20} = 1,048,576$$

- ▶ Weights get enormous, model becomes unstable, loss = NaN
- ▶ **Fix:** *Gradient clipping* — if gradient $>$ threshold, scale it down
 - `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`

Problem	Cause	Fix
Vanishing	Factors < 1	LSTM / GRU
Exploding	Factors > 1	Gradient clipping

- ▶ Vanilla RNNs squash everything through a single $\tanh$
- ▶ **Problem:** Old information gets overwritten by new information
- ▶ **What we need:**
 - A way to *selectively forget* irrelevant info
 - A way to *selectively remember* important info
 - A *separate memory channel* that doesn't get squashed
- ▶ This is exactly what **LSTM** and **GRU** do!

Next: LSTM — Long Short-Term Memory

Part 6: **LSTM** — Long Short-Term Memory

Hochreiter & Schmidhuber, 1997

- ▶ An LSTM is an RNN with a *smarter memory system*
- ▶ Instead of one hidden state, LSTM has **two**:
 - **Cell state** C_t : The long-term memory (like a notebook)
 - **Hidden state** h_t : The short-term/working memory
- ▶ The cell state is a *highway* for information — data flows through with minimal changes
- ▶ **Gates** control what enters and leaves the cell state

Analogy: Your Backpack

Cell state = your backpack (long-term storage)

Hidden state = what's in your hand right now

Gates = you deciding what to keep, throw away, or pull out

1. **Forget Gate** f_t : What should we *throw away* from memory?
 - Outputs values between 0 (forget completely) and 1 (keep everything)
 - $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$
2. **Input Gate** i_t : What *new information* should we store?
 - Decides which new values to add to memory
 - $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
3. **Output Gate** o_t : What should we *output* from memory?
 - Decides what part of memory to reveal as the hidden state
 - $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

All gates use sigmoid $\sigma \rightarrow$ outputs between 0 and 1 = “how much to allow through”

Step 1: Decide what to forget from old memory

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- ▶ Looks at the previous output h_{t-1} and current input x_t
- ▶ Outputs a number between 0 and 1 for each cell in C_{t-1}
- ▶ $f_t = 0$: completely forget this $f_t = 1$: keep this

Running Example

Reading “The cat sat on the **mat**”

When we reach “sat”, the forget gate might decide to forget grammatical details about “The” (article) since they’re no longer needed.

Step 2: Decide what new info to store

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{what to update})$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (\text{candidate values})$$

- ▶ i_t : decides *which* cells to update (0 or 1)
- ▶ $\tilde{C}_t$: proposes *new candidate values* (between -1 and 1)
- ▶ Together: only the important new info gets written to memory

Running Example

When reading “cat”, the input gate says “this is the subject — store it!”
The candidate creates a representation of “cat” to add to memory.

Step 3: Update the long-term memory

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

- ▶ $f_t \odot C_{t-1}$: **keep** old memories that passed the forget gate
- ▶ $i_t \odot \tilde{C}_t$: **add** new info that passed the input gate
- ▶ $\odot$ means element-wise multiplication

Analogy

Like updating your notebook:

1. Cross out notes you don't need anymore (forget gate)
2. Write down new important facts (input gate)

The notebook (C_t) now has the most useful information!

Step 4: Decide what to output

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

- ▶ Not everything in memory is relevant right now
- ▶ The output gate selects which parts of cell state to reveal
- ▶ h_t is what gets passed to the next step and used for prediction

Running Example

After reading “The cat sat on the”:

Memory (C_t) stores: subject=cat, action=sitting, preposition=on

Output (h_t) might emphasize: “something is on something” → predict “mat”

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Forget Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Input Gate

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

Candidate

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Cell State Update

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Output Gate

$$h_t = o_t \odot \tanh(C_t)$$

Hidden State

- **Why this fixes vanishing gradients:** The cell state C_t has an *additive* update (+), not multiplicative. Gradients can flow through the + without shrinking!

Vanilla RNN:

- ▶ Like a **winding road** through mountains
- ▶ Information gets distorted at every turn
- ▶ Old info fades away quickly
- ▶ Hard for gradients to travel far

LSTM:

- ▶ Like a **highway** with on/off ramps
- ▶ Cell state = the highway (fast, direct)
- ▶ Gates = the on-ramps and off-ramps
- ▶ Information can travel long distances unchanged

LSTM doesn't guarantee no vanishing gradients, but makes it much easier to learn long-range dependencies.

Part 7: **GRU** — Gated Recurrent Unit

Cho et al., 2014

- ▶ GRU is like a *simplified LSTM*
- ▶ Instead of 3 gates + cell state, GRU has only **2 gates**:
 - ▶ **Reset Gate** r_t : How much of the past to forget
 - ▶ **Update Gate** z_t : How much of the new info to use
- ▶ **No separate cell state** — hidden state does everything
- ▶ Fewer parameters → faster training

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Reset Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Update Gate

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Candidate

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Final State

- ▶ **Reset gate** r_t : When $r_t \approx 0$, ignore previous hidden state $\rightarrow$ “start fresh”
- ▶ **Update gate** z_t : Blends old and new
 - $z_t \approx 0$: keep old state (don't update)
 - $z_t \approx 1$: use new candidate (full update)
- ▶ Notice: $h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$ is like a *weighted average*!

Running Example with GRU

Processing “The cat sat on the [?]”:

- ▶ Reading “The”: update gate stores article context
- ▶ Reading “cat”: reset gate partially resets, update gate stores subject
- ▶ Reading “sat”: update gate adds action to memory
- ▶ Reading “on”: update gate adds preposition
- ▶ Reading “the”: update gate keeps everything, predict next word

Key insight: The update gate acts like *both* the forget gate and input gate of LSTM combined into one!

Feature	LSTM	GRU
Number of gates	3 (forget, input, output)	2 (reset, update)
Separate cell state	✓ Yes	✗ No
Parameters	More	Fewer
Training speed	Slower	Faster
Long sequences	Better	Good
Short sequences	Good	Often better
Memory usage	Higher	Lower

- ▶ **Rule of thumb:** Try GRU first (faster). Switch to LSTM if you need better long-range memory.
- ▶ In practice, performance is often *very similar*

NLP Tasks:

- ▶ Language modeling
- ▶ Machine translation
- ▶ Sentiment analysis
- ▶ Text generation

Audio & Speech:

- ▶ Speech recognition
- ▶ Music generation

Time Series:

- ▶ Weather forecasting
- ▶ Stock price prediction
- ▶ Anomaly detection

Video:

- ▶ Action recognition
- ▶ Video captioning

But: RNNs/LSTMs/GRUs are being replaced by Transformers in many of these tasks!

- ✗ **Sequential processing:** Must process words one at a time — *can't parallelize*
 - GPUs are great at parallel processing, but RNNs can't take advantage of this
- ✗ **Still struggle with very long sequences** (even LSTMs)
 - LSTM helps, but 1000+ word documents are still challenging
- ✗ **Information bottleneck** in seq2seq:
 - The entire input sequence must be compressed into one fixed-size vector
 - Like summarizing an entire book in one sentence!

Can we let the model “look back” at any part of the input? → Attention!

Part 8: LSTM in PyTorch

```
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
                  num_classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim,
                            batch_first=True,
                            bidirectional=True)

        # bidirectional => hidden_dim * 2
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x):
        emb = self.embedding(x)
        out, (h_n, c_n) = self.lstm(emb)
        # h_n: (2, batch, hidden) for bidirectional
        # Concatenate forward and backward final states
        h_cat = torch.cat([h_n[0], h_n[1]], dim=1)
        logits = self.fc(h_cat)
        return logits
```

- ▶ `nn.LSTM` replaces `nn.RNN` — same interface!
- ▶ Returns both h_n (hidden state) and c_n (cell state)
- ▶ `bidirectional=True`: reads forward AND backward

```
# Example: sentiment analysis on movie reviews
model = SentimentLSTM(vocab_size=5000, embed_dim=64,
                      hidden_dim=128, num_classes=2)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(10):
    for batch_x, batch_y in train_loader:
        logits = model(batch_x)
        loss = loss_fn(logits, batch_y)
        optimizer.zero_grad()
        loss.backward()
        # Gradient clipping prevents exploding gradients
        torch.nn.utils.clip_grad_norm_(
            model.parameters(), max_norm=1.0)
        optimizer.step()

# Evaluate
model.eval()
with torch.no_grad():
    preds = model(val_x).argmax(dim=1)
    acc = (preds == val_y).float().mean()
    print(f"Epoch {epoch}: Accuracy = {acc:.2%}")
```

Switching to GRU: Just One Line!

```
class SentimentGRU(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
                  num_classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        # Only change: nn.LSTM -> nn.GRU
        self.gru = nn.GRU(embed_dim, hidden_dim,
                           batch_first=True,
                           bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x):
        emb = self.embedding(x)
        out, h_n = self.gru(emb) # No cell state!
        h_cat = torch.cat([h_n[0], h_n[1]], dim=1)
        return self.fc(h_cat)
```

Key Differences in Code

LSTM: `out, (h_n, c_n) = self.lstm(emb)` ← returns cell state

GRU: `out, h_n = self.gru(emb)` ← no cell state

Everything else is identical! PyTorch makes switching easy.

Part 9: **The Attention Mechanism**

- ▶ In seq2seq (e.g., translation), the encoder compresses the *entire input* into **one vector**
- ▶ This single vector must capture everything about a possibly very long sentence

The Problem

Translating a 50-word paragraph from English to Arabic:

50 words $\xrightarrow{\text{encoder}}$ **1 vector** $\xrightarrow{\text{decoder}}$ **Arabic translation**

That one vector must somehow remember all 50 words!

It's like summarizing an entire book in a single sentence.

- ▶ **Result:** Performance drops sharply on longer sentences
- ▶ **We need:** A way to let the decoder “look back” at the entire input

Bahdanau et al., 2015

- ▶ Instead of using only the *last* encoder hidden state...
- ▶ Let the decoder *look at ALL encoder hidden states*!
- ▶ At each decoding step, the decoder decides **which input words to focus on**

Analogy: Reading a Test Question

When answering a question about a long passage:

- ▶ You don't try to remember the whole passage in your head
- ▶ Instead, you **look back** at the relevant parts
- ▶ For different questions, you focus on **different parts**

This is exactly what attention does!

At each decoder step t :

1. **Score:** Compare the decoder state s_t with each encoder hidden state h_i

$$e_{t,i} = \text{score}(s_t, h_i)$$

2. **Normalize:** Convert scores to weights using softmax

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}$$

3. **Weight:** Compute weighted sum of encoder states

$$c_t = \sum_i \alpha_{t,i} \cdot h_i$$

4. **Use:** Feed context vector c_t to the decoder along with s_t

The weights $\alpha_{t,i}$ tell us: “*how much attention to pay to input word i* ”

Translating “The cat sat on the mat” to French

When generating “**Le**”:

$\alpha = [0.9, 0.02, 0.02, 0.02, 0.02, 0.02] \leftarrow$ focuses on “The”

When generating “**chat**”:

$\alpha = [0.05, 0.85, 0.03, 0.02, 0.03, 0.02] \leftarrow$ focuses on “cat”

When generating “**assis**”:

$\alpha = [0.02, 0.05, 0.80, 0.05, 0.03, 0.05] \leftarrow$ focuses on “sat”

When generating “**tapis**”:

$\alpha = [0.02, 0.02, 0.03, 0.05, 0.03, 0.85] \leftarrow$ focuses on “mat”

- ▶ Each output word *attends to different* input words
- ▶ The attention weights form a **soft alignment** between input and output

How to compute $e_{t,i} = \text{score}(s_t, h_i)$?

Name	Formula	Notes
Dot product	$s_t^T h_i$	Simplest, fast
Scaled dot product	$\frac{s_t^T h_i}{\sqrt{d}}$	Prevents large values
Additive (Bahdanau)	$v^T \tanh(W_1 s_t + W_2 h_i)$	More expressive
General (Luong)	$s_t^T W h_i$	Learnable similarity

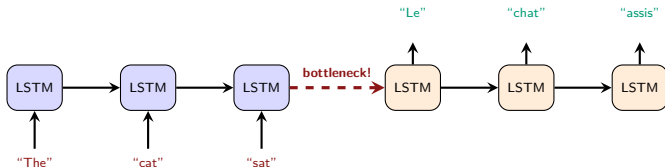
- ▶ *Scaled dot product* is most common today (used in Transformers!)
- ▶ Scaling by $\sqrt{d}$ prevents softmax from becoming too “sharp”

- ✓ **Solves the bottleneck:** No need to compress everything into one vector
- ✓ **Handles long sequences:** Can attend to any position, near or far
- ✓ **Interpretable:** Attention weights show what the model “looks at”
- ✓ **Improves gradient flow:** Creates shortcuts for gradients to flow directly from output to input

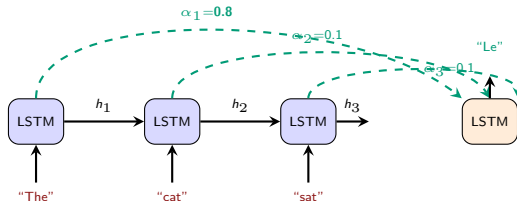
Fun Fact

Before attention, machine translation quality dropped sharply for sentences longer than 20 words. With attention, it handles 50+ word sentences easily!

Part 10: **Attention in Seq2Seq**



- **Problem:** Only the **last encoder state** is passed to the decoder
- All information about "The", "cat", "sat" must fit in one vector
- Longer sentences → more information loss



- ▶ The decoder can now *look at all encoder states* h_1, h_2, h_3
- ▶ When generating "Le" (translating "The"), it pays most attention to h_1
- ▶ When generating "chat" (translating "cat"), it would focus on h_2
- ▶ **Each output step gets its own custom context vector!**

For each decoder step t :

1. **Compute scores:** $e_{t,i} = s_t^T \cdot h_i$ for all encoder states
 2. **Softmax:** $\alpha_{t,i} = \text{softmax}(e_{t,i})$
 3. **Context vector:** $c_t = \sum_i \alpha_{t,i} \cdot h_i$
 4. **Combine:** Feed $[c_t; s_t]$ to the output layer
 5. **Predict:** $\hat{y}_t = \text{softmax}(W_o[c_t; s_t])$
- ▶ The LSTM encoder produces all hidden states $h_1, \dots, h_n$
 - ▶ The LSTM decoder uses attention to “focus” on the right ones
 - ▶ This combination is called *Attention-based Seq2Seq* or *Bahdanau Attention*

English → French Translation: Attention Matrix

	The	cat	sat	mat
Le	0.85	0.05	0.05	0.05
chat	0.05	0.80	0.10	0.05
assis	0.03	0.07	0.82	0.08
tapis	0.02	0.03	0.05	0.90

- ▶ The diagonal pattern shows word-to-word alignment
- ▶ In real languages, alignment isn't always diagonal
 - French often reorders adjective-noun: “red cat” → “chat rouge”
- ▶ Attention learns these patterns automatically!

- ▶ Attention + LSTM is great, but there's still a bottleneck:
 - The LSTM encoder is still *sequential* — word by word
 - We can't process words in parallel
 - Training is slow for long sequences
- ▶ **A radical question** (Vaswani et al., 2017):

**“What if we throw away the RNN entirely
and use *ONLY* attention?”**

This is the *Transformer* — and it changed everything!

Part 11: **Self-Attention**

- ▶ **Regular attention** (in seq2seq): decoder attends to *encoder* states
 - Cross-attention: two different sequences interact
- ▶ **Self-attention**: a sequence attends to *itself*!
 - Each word looks at **all other words** in the same sentence
 - Every word gets a new representation based on its context

Self-Attention Example

“The **cat** sat on the mat because **it** was tired”

Self-attention helps “it” figure out that “it” = “cat”
by attending strongly to “cat” even though it’s far away!

- Self-attention uses three vectors for each word:

Analogy: Searching a Library

Query (Q) = Your question: “What is the subject of this sentence?”

Key (K) = Book titles: Each word has a “label” describing what info it holds

Value (V) = Book contents: The actual information each word carries

Process:

1. Compare your **query** to every **key** (which books match?)
2. Get **scores** (how relevant is each book?)
3. Read the **values** of the most relevant books (weighted sum)

- For each word embedding x_i , create three vectors:

$$q_i = W_Q \cdot x_i \quad (\text{Query: "What am I looking for?"})$$

$$k_i = W_K \cdot x_i \quad (\text{Key: "What do I contain?"})$$

$$v_i = W_V \cdot x_i \quad (\text{Value: "Here is my information"})$$

- W_Q , W_K , W_V are **learned weight matrices**
- Each word gets its own Q, K, V — but the weight matrices are shared

Word	Q	K	V
"The"	$q_1 = W_Q x_1$	$k_1 = W_K x_1$	$v_1 = W_V x_1$
"cat"	$q_2 = W_Q x_2$	$k_2 = W_K x_2$	$v_2 = W_V x_2$
"sat"	$q_3 = W_Q x_3$	$k_3 = W_K x_3$	$v_3 = W_V x_3$

The attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Step by step:

1. QK^T : Dot product of every query with every key $\rightarrow$ similarity scores
 2. $\nabla \cdot \sqrt{d_k}$: Scale down to prevent very large numbers
(if $d_k = 64$, we divide by $\sqrt{64} = 8$)
 3. softmax: Convert to probabilities (sum to 1)
 4. $\times V$: Weighted sum of values
- **Result:** Each word gets a new representation that *incorporates context* from all other words!

Self-Attention on “The cat sat”

Step 1: Compute scores ($Q \cdot K^T$):

	The	cat	sat
The	2.1	0.5	0.3
cat	0.4	3.2	1.8
sat	0.2	1.5	2.9

Step 2: Scale by $\sqrt{d_k}$ and softmax:

	The	cat	sat
The	0.65	0.20	0.15
cat	0.10	0.55	0.35
sat	0.08	0.32	0.60

Step 3: New “cat” = $0.10 \cdot v_{\text{The}} + 0.55 \cdot v_{\text{cat}} + 0.35 \cdot v_{\text{sat}}$

Now “cat” knows it’s a cat *that sat*!

Property	RNN / LSTM	Self-Attention
Path length (word 1 to word n)	$O(n)$ steps	$O(1)$ direct
Parallelizable	✗ Sequential	✓ Fully parallel
Long-range dependencies	Hard (even LSTM)	Easy (direct connection)
Training speed	Slow	Fast (GPU-friendly)
Computation per layer	$O(n \cdot d^2)$	$O(n^2 \cdot d)$

- ▶ In RNNs, to connect word 1 and word 100, information passes through 99 steps
- ▶ In self-attention, word 1 and word 100 are *directly connected*!
- ▶ Trade-off: self-attention is $O(n^2)$ in sequence length (costly for very long sequences)

Part 12: **The Transformer Architecture**

Vaswani et al., 2017 — Google Brain

- ▶ Proposed a model that uses **only attention** — no RNNs, no convolutions
- ▶ Called the *Transformer*
- ▶ Has become the **dominant architecture** in AI:
 - NLP: BERT, GPT, ChatGPT, LLaMA
 - Vision: ViT (Vision Transformer)
 - Audio: Whisper
 - Multimodal: CLIP, GPT-4

Fun Fact

The original paper title “Attention Is All You Need” was almost rejected. Today it has over 100,000 citations — one of the most influential AI papers ever!

- ▶ One attention head captures one type of relationship
- ▶ **Multi-head:** Run attention *multiple times in parallel*
- ▶ Each “head” learns to focus on *different aspects*:
 - Head 1: “Who is the subject?” (cat $\leftrightarrow$ sat)
 - Head 2: “Where did it happen?” (sat $\leftrightarrow$ mat)
 - Head 3: “What grammar pattern?” (The $\leftrightarrow$ cat)

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- ▶ Each head has its own W^Q , W^K , W^V (smaller dimensions)
- ▶ Results are concatenated and projected back
- ▶ Typical: $h = 8$ heads, $d_k = d_{\text{model}}/h = 512/8 = 64$

- ▶ Self-attention treats input as a *set*, not a sequence!
- ▶ “The cat sat” and “sat cat the” would give the same result
- ▶ But word order matters in language!
- ▶ **Solution: Positional Encoding** — add position information to each word

$$\text{input}_i = \text{word_embedding}_i + \text{position_encoding}_i$$

- ▶ **Sinusoidal encoding** (original Transformer):

$$PE(pos, 2k) = \sin\left(\frac{pos}{10000^{2k/d}}\right)$$
$$PE(pos, 2k + 1) = \cos\left(\frac{pos}{10000^{2k/d}}\right)$$

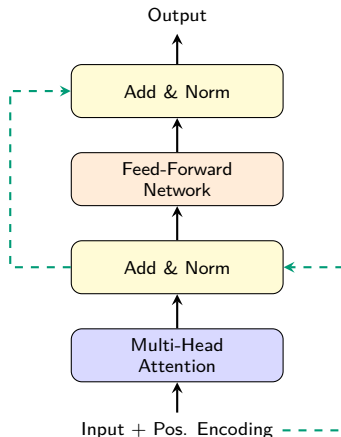
- ▶ Each position gets a unique pattern — the model learns to use it

Think of It Like a Clock

- ▶ The second hand moves fast (high frequency) → encodes fine position
- ▶ The minute hand moves slower → encodes medium-range position
- ▶ The hour hand moves slowest → encodes coarse position

Similarly, different dimensions of the positional encoding oscillate at different frequencies, giving each position a unique “fingerprint.”

- ▶ **Why not just use $[1, 2, 3, \dots]$?**
 - Those numbers grow unboundedly
 - Sin/cos values stay between -1 and 1 — same scale as embeddings
 - The model can also learn *relative* positions (distance between words)



Each encoder block has:

1. **Multi-Head Self-Attention**
 - Each word attends to all words
2. **Add & Norm** (residual + layer norm)
 - Dashed arrows = skip connections
 - Helps gradient flow (like ResNet!)
3. **Feed-Forward Network**
 - 2-layer MLP applied to each word independently
 - Adds nonlinearity and capacity
4. **Add & Norm** again

Stack $N = 6$ of these blocks!

Residual Connection (Skip Connection):

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- ▶ The input x is *added* to the sublayer's output (then normalized)
- ▶ Same idea as ResNet: helps gradients flow through deep networks
- ▶ Without skip connections, 6-block Transformers would be very hard to train

Layer Normalization:

- ▶ Normalizes across features (not across batch like batch norm)
- ▶ Stabilizes training by keeping activations in a consistent range
- ▶ Applied after each sub-layer (attention + feed-forward)

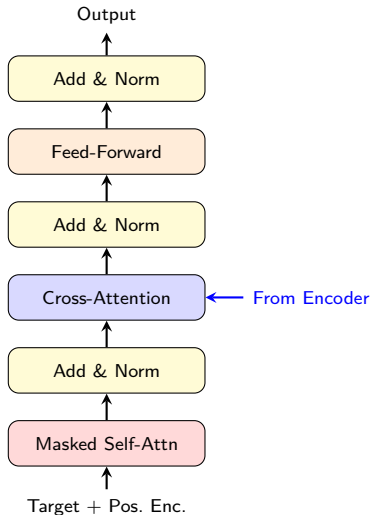
- ▶ Applied to each position **independently** (no interaction between words)
- ▶ Two linear layers with ReLU in between:

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2$$

- ▶ Inner dimension is typically **4x** the model dimension:
 - $d_{\text{model}} = 512 \rightarrow d_{\text{ff}} = 2048$
- ▶ **Purpose:** Adds processing power beyond what attention alone provides

Analogy

Self-attention is like a **group discussion** — words share information.
FFN is like **individual thinking** — each word processes what it learned.



Decoder has 3 sub-layers:

1. **Masked Self-Attention** — can only attend to *previous* positions; prevents “cheating” by looking at future words
2. **Cross-Attention** — Q from decoder, K and V from *encoder*; this is how the decoder “reads” the input
3. **Feed-Forward Network** — same as in encoder

- ▶ When generating text, you can't look at future words (they don't exist yet!)
- ▶ **Masking:** Set future positions to $-\infty$ before softmax

	The	cat	sat	on
The	✓	$-\infty$	$-\infty$	$-\infty$
cat	✓	✓	$-\infty$	$-\infty$
sat	✓	✓	✓	$-\infty$
on	✓	✓	✓	✓

- ▶ After softmax, $-\infty \rightarrow 0$ (zero attention weight)
- ▶ “cat” can attend to “The” and itself, but NOT to “sat” or “on”
- ▶ This is called *causal masking* — used in GPT, LLaMA, etc.

Attention is all you need

Vaswani et al, NeurIPS 2017

The original “Attention Is All You Need” architecture:

- ▶ **Left:** Encoder (6 blocks)
 - Self-attention + FFN
 - Processes input all at once (parallel!)
- ▶ **Right:** Decoder (6 blocks)
 - Masked self-attention + cross-attention + FFN
 - Generates output one token at a time
- ▶ **Top:** Linear + Softmax → word probabilities
- ▶ **Bottom:** Input/output embeddings + positional encoding

Variant	Architecture	Example	Task
Encoder-only	Just encoder	<i>BERT</i>	Classification, QA
Decoder-only	Just decoder	<i>GPT</i> , LLaMA	Text generation
Encoder-decoder	Both	<i>T5</i> , BART	Translation, summary

- ▶ **BERT** (encoder-only): Reads text bidirectionally, great for understanding
- ▶ **GPT** (decoder-only): Generates text left-to-right, powers ChatGPT
- ▶ **T5** (encoder-decoder): Converts any task to text-to-text
- ▶ Most modern LLMs are *decoder-only* (GPT-4, LLaMA, Claude, Gemini)

Part 13: **Transformer in PyTorch**

```
import torch
import torch.nn as nn
import math

class SelfAttention(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.d_k = d_model

    def forward(self, x):
        Q = self.W_Q(x) # (batch, seq, d_model)
        K = self.W_K(x)
        V = self.W_V(x)
        # Scaled dot-product attention
        scores = torch.matmul(Q, K.transpose(-2, -1))
        scores = scores / math.sqrt(self.d_k)
        weights = torch.softmax(scores, dim=-1)
        output = torch.matmul(weights, V)
        return output
```

- ▶ 3 linear layers create Q, K, V from the input

- ▶ Matrix multiply $\rightarrow$ scale $\rightarrow$ softmax $\rightarrow$ weighted sum

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(
            d_model, n_heads, batch_first=True)
        self.norm1 = nn.LayerNorm(d_model)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model))
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # Self-attention + residual + norm
        attn_out, _ = self.attention(x, x, x)
        x = self.norm1(x + self.dropout(attn_out))
        # Feed-forward + residual + norm
        ff_out = self.ff(x)
        x = self.norm2(x + self.dropout(ff_out))
        return x
```



```
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, d_model=128,
                  n_heads=4, n_layers=2, n_classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_enc = nn.Embedding(512, d_model)
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_heads, d_model*4)
            for _ in range(n_layers)])
        self.fc = nn.Linear(d_model, n_classes)

    def forward(self, x):
        seq_len = x.size(1)
        positions = torch.arange(seq_len, device=x.device)
        x = self.embedding(x) + self.pos_enc(positions)
        for block in self.blocks:
            x = block(x)
        # Use mean of all positions for classification
        x = x.mean(dim=1)
        return self.fc(x)
```

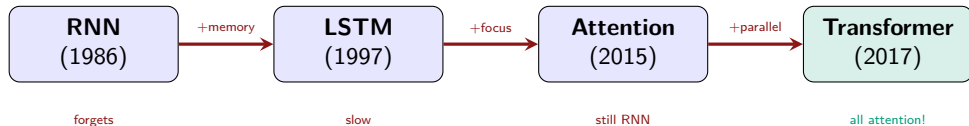
- ▶ Word embedding + positional embedding added together
- ▶ Stack multiple Transformer blocks

```
model = TransformerClassifier(  
    vocab_size=5000, d_model=128,  
    n_heads=4, n_layers=2, n_classes=2)  
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)  
loss_fn = nn.CrossEntropyLoss()  
  
for epoch in range(10):  
    for batch_x, batch_y in train_loader:  
        logits = model(batch_x)  
        loss = loss_fn(logits, batch_y)  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()
```

Compared to LSTM

Same training loop! The only difference is the model architecture. PyTorch makes it easy to swap between RNN, LSTM, GRU, and Transformer — the training code stays the same.

Part 14: **Summary & The Big Picture**



- ▶ Each step fixed a problem from the previous approach
- ▶ **RNN:** Sequential memory, but forgets long-range info
- ▶ **LSTM/GRU:** Better memory via gates, but still sequential
- ▶ **Attention:** Can focus on any part of input, but still uses RNN backbone
- ▶ **Transformer:** Attention only, fully parallel, dominates all of AI

	RNN	LSTM	GRU	Transformer
Memory	Short	Long	Long	Unlimited
Parallelizable	✗	✗	✗	✓
Training speed	Slow	Slow	Medium	Fast
Parameters	Few	Medium	Medium	Many
Long sequences	✗	✓	✓	✓✓
Interpretable	✗	✗	✗	✓ (attn)
Year	1986	1997	2014	2017

- ▶ Transformers win on most dimensions, but use more memory ($O(n^2)$)
- ▶ RNNs/LSTMs are still used for some real-time or edge applications

Predicting “mat” from “The cat sat on the ____”

RNN: Reads word by word. By “the” (word 5), might have forgotten “cat” was the subject.

LSTM: Reads word by word. Gates help remember “cat sat on” even at step 5.

Attention + LSTM: At prediction time, looks back at ALL words — especially “on” and “the” — to predict “mat”.

Transformer: Processes ALL 5 words simultaneously. Each word sees every other word instantly. Predicts “mat” with full context.

Built on Transformers:

- ▶ *ChatGPT* (GPT-4, decoder-only)
- ▶ *BERT* (encoder-only, text understanding)
- ▶ *T5* (encoder-decoder, text-to-text)
- ▶ *ViT* (Vision Transformer)
- ▶ *Whisper* (speech recognition)
- ▶ *CLIP* (vision + language)
- ▶ *Stable Diffusion* (image generation)

Where RNNs still appear:

- ▶ Edge devices (small models)
- ▶ Real-time streaming (character by character)
- ▶ Some time-series forecasting
- ▶ Legacy systems
- ▶ When training data is very small

But Transformers are taking over even these areas!

1. **RNNs** process sequences by maintaining a hidden state — but forget long-range dependencies
2. **LSTMs** solve this with gates (forget, input, output) and a cell state “highway”
3. **GRUs** are a simpler LSTM with 2 gates — often comparable performance
4. **Attention** lets models focus on relevant parts of the input — no bottleneck
5. **Self-attention** lets each word attend to all other words in the same sequence
6. **Transformers** = self-attention + FFN + positional encoding, fully parallel
7. Three Transformer variants: *encoder-only* (BERT), *decoder-only* (GPT), *encoder-decoder* (T5)

Tomorrow: We'll use pre-trained Transformers (BERT) for real NLP tasks!

1. What is the hidden state in an RNN?
2. Why do vanilla RNNs struggle with long sequences?
3. Name the three gates in an LSTM and what each does.
4. How is a GRU different from an LSTM?
5. What problem does attention solve in seq2seq models?
6. What are Q, K, V in self-attention?
7. Why do Transformers need positional encoding?
8. What is masked self-attention and why is it needed?
9. Name one encoder-only, one decoder-only, and one encoder-decoder Transformer.
10. Why can Transformers be trained faster than RNNs?

Credits

KAUST Academy

King Abdullah University of Science and Technology

RNNs, Attention & Transformers content authored for the
KAUST–Mawhiba IOAI 2026 Training Program

Transformer architecture figure from Vaswani et al., 2017

Week 3 · Day 17